

Supervised and Unsupervised Machine Learning

Mamta Bhattarai

MBUST

July 7, 2026

Unit Overview: Supervised Learning Algorithms

In this unit, we will study the following **supervised learning algorithms**:

- **Linear Regression**: Predicts continuous values using best-fit line.
- **Logistic Regression**: Classification using probabilistic decision boundary.
- **k-Nearest Neighbours (k-NN)**: Classifies based on nearest data points.
- **Decision Trees**: Rule-based hierarchical splitting of data.
- **Naïve Bayes Classifier**: Probabilistic model using Bayes theorem.
- **Support Vector Machine (SVM)**: Maximum margin classifier for optimal separation.

Unit Overview: Unsupervised Learning Algorithms

We will also study the following **unsupervised learning algorithms**:

- **K-means Clustering**: Groups data into K clusters based on similarity.
- **Hierarchical Clustering**: Builds tree-like structure of clusters.
- **DBSCAN**: Density-based clustering for arbitrary shaped clusters.
- **Anomaly Detection**: Identifies rare or unusual patterns in data.

What is Linear Regression?

- Linear Regression is a supervised learning algorithm.
- It predicts continuous numerical values.
- It models the relationship between input and output variables using a straight line.

Examples

- House price prediction
- Salary estimation
- Sales forecasting
- Temperature prediction

Simple Linear Regression

Simple Linear Regression uses one independent variable to predict one dependent variable.

$$y = \beta_0 + \beta_1 x$$

where

- x : Independent variable
- y : Dependent variable
- β_0 : Intercept
- β_1 : Slope (coefficient)

Example Dataset

Area (sq ft)	Price (\$1000)
1000	200
1500	280
2000	360
2500	430

As the area increases, the house price also increases.

Components of Regression Equation

$$y = 50 + 3x$$

Intercept (β_0)

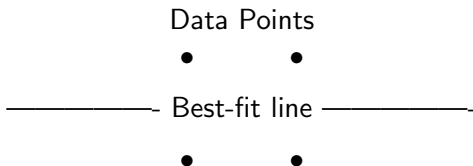
- Value of y when $x = 0$.

Slope (β_1)

- Change in y for one unit increase in x .

Goal of Linear Regression

The objective is to find the best fitting line that minimizes prediction error.



Prediction Error

Predicted value:

$$\hat{y}$$

Residual (Error):

$$Error = y - \hat{y}$$

Residuals measure the difference between actual and predicted values.

Cost Function

Linear Regression minimizes Mean Squared Error (MSE).

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Smaller MSE indicates better performance.

Multiple Linear Regression

Multiple Linear Regression uses more than one feature.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

Examples of features:

- Area
- Number of bedrooms
- Age of house
- Location

Assumptions of Linear Regression

- 1 Linear relationship between variables.
- 2 Independent observations.
- 3 Constant variance of errors.
- 4 Normally distributed residuals.
- 5 No multicollinearity.
- 6 No significant outliers.

Evaluation Metrics

Mean Absolute Error (MAE)

$$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

Mean Squared Error (MSE)

$$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{MSE}$$

Coefficient of Determination

$$R^2$$

Coefficient of Determination (R^2)

R^2 measures how well the regression model explains the variability in the data.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

where

- $SS_{res} = \sum (y_i - \hat{y}_i)^2$: Residual Sum of Squares
- $SS_{tot} = \sum (y_i - \bar{y})^2$: Total Sum of Squares
- $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$: Mean of the actual values

Interpretation

- $R^2 = 1$: Perfect fit
- $R^2 = 0$: Model explains none of the variance
- Higher R^2 indicates a better model fit

Advantages

- Simple and easy to understand.
- Computationally efficient.
- Easy to interpret.
- Works well for linear relationships.

Limitations

- Sensitive to outliers.
- Assumes linearity.
- Performs poorly on nonlinear data.
- Can be affected by multicollinearity.

Applications

- House price prediction
- Sales forecasting
- Healthcare analytics
- Economic modeling
- Demand estimation
- Risk assessment

Summary

- Linear Regression predicts continuous values.
- It models the relationship between variables using a straight line.
- The objective is to minimize prediction error.
- Common evaluation metrics include MAE, MSE, RMSE, and R^2 .
- It is one of the most widely used machine learning algorithms.

What is Logistic Regression?

- Logistic Regression is a supervised learning algorithm.
- Used for classification problems.
- Predicts the probability that an instance belongs to a class.
- Most commonly used for binary classification.

Examples

- Spam detection
- Disease diagnosis
- Credit card fraud detection
- Customer churn prediction

Binary Classification

Two possible outcomes:

Positive Class (1)

Negative Class (0)

Examples:

- Yes / No
- True / False
- Spam / Not Spam
- Pass / Fail

Why Not Linear Regression?

Linear regression may predict values outside the interval $[0, 1]$.

Logistic regression converts predictions into probabilities between 0 and 1.

$$0 \leq P(Y = 1|X) \leq 1$$

Hence, it is suitable for classification tasks.

The Sigmoid Function

Logistic regression uses the sigmoid function:

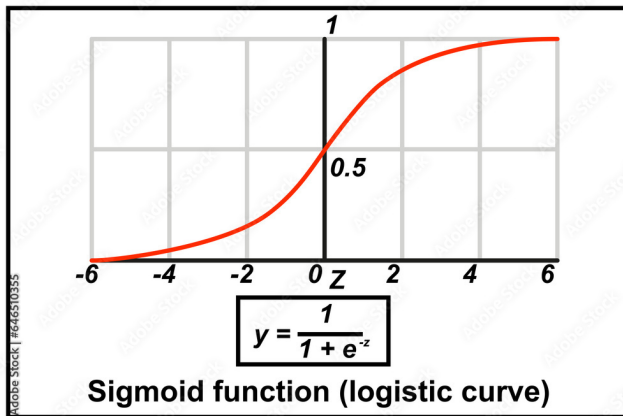
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where

$$z = \beta_0 + \beta_1 x$$

The output is always between 0 and 1.

Shape of the Sigmoid Function



- S-shaped curve.
- Converts any real number into a probability.

Decision Boundary

Predicted probability:

$$P(Y = 1|X)$$

Classification rule:

$$\hat{y} = \begin{cases} 1, & P(Y = 1|X) \geq 0.5 \\ 0, & P(Y = 1|X) < 0.5 \end{cases}$$

Threshold value is usually 0.5.

Log Odds (Logit)

The logit function is

$$\log \left(\frac{p}{1-p} \right) = \beta_0 + \beta_1 x$$

where

- p is the probability of belonging to class 1.
- $\frac{p}{1-p}$ represents the odds.

Cost Function

Cross-entropy is a loss function used to measure how different the predicted probabilities are from the true labels. Logistic regression uses Binary Cross-Entropy Loss:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Goal:

Minimize the loss function.

Training Process

- 1 Collect training data.
- 2 Initialize parameters.
- 3 Compute predictions.
- 4 Calculate loss.
- 5 Update parameters using gradient descent.
- 6 Repeat until convergence.

Common evaluation metrics:

- Accuracy
- Precision
- Recall
- F1-Score
- ROC Curve
- AUC

Confusion Matrix

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

- TP: True Positive
- TN: True Negative
- FP: False Positive
- FN: False Negative

Performance Metrics

Common evaluation metrics:

- **Accuracy**

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision**

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall (Sensitivity)**

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-Score**

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **ROC Curve**

ROC plots: TPR vs FPR

where

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN}$$

Advantages

- Simple and efficient.
- Easy to interpret.
- Produces probability estimates.
- Works well for linearly separable data.

Limitations

- Assumes linear relationship between features and log odds.
- Sensitive to outliers.
- May perform poorly on complex nonlinear data.
- Requires sufficient training data.

Applications

- Email spam detection
- Medical diagnosis
- Credit scoring
- Fraud detection
- Sentiment analysis
- Customer churn prediction

Summary

- Logistic regression is used for classification.
- It predicts probabilities using the sigmoid function.
- Output values lie between 0 and 1.
- Binary cross-entropy is used as the loss function.
- Performance is evaluated using accuracy, precision, recall, F1-score, ROC, and AUC.

Support Vector Machine (SVM): Conceptual Foundation

Support Vector Machine (SVM) is a powerful **supervised learning algorithm** used for both classification and regression, though it is most commonly applied to classification problems.

Core Idea:

- SVM finds an **optimal separating hyperplane** that divides data points of different classes.
- The optimality is defined by the **maximum margin principle**.
- The margin is the distance between the hyperplane and the nearest data points from each class.

Key Principle: Maximum Margin

- Among all possible separating hyperplanes, SVM selects the one that maximizes the margin.
- This improves **generalization ability** and reduces overfitting.

Support Vectors:

- The closest data points to the decision boundary.
- These points are critical because they fully determine the position of the hyperplane.
- Removing other points does not change the decision boundary.

Mathematical Form:

$$w^T x + b = 0$$

represents the decision boundary separating the classes.

Support Vector Machine (SVM): Geometric View

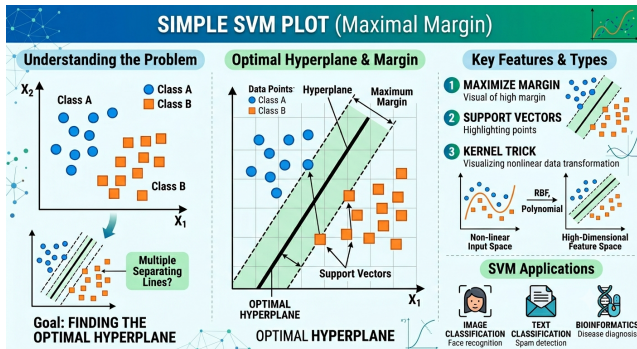


Figure: SVM: maximum margin hyperplane, margins, and support vectors

Key Idea:

- The hyperplane separates two classes.
- Margins define the maximum separation region.
- Support vectors are the closest points that define the boundary.

Support Vector Machine (SVM): Intuition

SVM is a **maximum margin classifier** that finds the best separating hyperplane between two classes.

- Goal: Find hyperplane $w^T x + b = 0$
- Maximize the margin between two classes
- Support vectors are the closest points that define the boundary

Decision boundary:

$$w^T x + b = 0$$

Margins:

$$w^T x + b = +1 \quad \text{and} \quad w^T x + b = -1$$

SVM Optimization Problem

Support Vector Machine is formulated as a **convex optimization problem** that finds the optimal separating hyperplane with maximum margin.

Hard Margin SVM (Linearly Separable Data):

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2} \|w\|^2 \\ \text{subject to} \quad & y_i(w^T x_i + b) \geq 1, \quad \forall i \end{aligned}$$

Key Idea:

- Minimizing $\|w\|^2$ maximizes the margin
- Constraints enforce correct classification

Soft Margin SVM (Practical Case)

For non-linearly separable or noisy data, SVM introduces slack variables ξ_i .

$$\begin{aligned} \min_{w, b, \xi} \quad & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \\ \text{subject to} \quad & y_i(w^T x_i + b) \geq 1 - \xi_i, \\ & \xi_i \geq 0, \quad \forall i \end{aligned}$$

Key Idea:

- ξ_i allows margin violation / misclassification
- C controls trade-off between margin and error
- Large $C \rightarrow$ low bias, high variance
- Small $C \rightarrow$ wider margin, more tolerance

SVM: Margin Concept

The geometric margin is:

$$\text{Margin} = \frac{2}{\|w\|}$$

Key Insight:

- Maximizing margin $\Leftrightarrow$ minimizing $\|w\|$
- Larger margin improves generalization

SVM Optimization Problem

Goal: Find the hyperplane with the maximum margin.

For linearly separable data, SVM solves

$$\min_{w,b} \frac{1}{2} \|w\|^2$$

subject to

$$y_i(w^T x_i + b) \geq 1, \quad i = 1, \dots, n.$$

- Minimize $\|w\| \Rightarrow$ maximize the margin.
- Constraints ensure every training sample is correctly classified.
- This is a constrained optimization problem.

How is the Optimization Solved?

Optimization Process

SVM Optimization → Lagrange Multipliers

→ Quadratic Programming (QP)

→ SMO

- **Lagrange Multipliers:** Convert the constrained optimization problem into its dual form.
- **Quadratic Programming (QP):** The dual optimization has a quadratic objective with linear constraints.
- **SMO (Sequential Minimal Optimization):** Efficiently solves the QP by updating two Lagrange multipliers at a time.

Result:

- Global optimum (convex optimization)
- Only support vectors determine the decision boundary

From Primal to Dual (SVM Derivation)

1. Primal Problem

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1$$

2. Lagrangian Formulation

$$L(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^n \alpha_i [y_i(w^T x_i + b) - 1]$$
$$\alpha_i \geq 0$$

3. Stationarity Conditions

$$\frac{\partial L}{\partial w} = 0 \Rightarrow w = \sum_{i=1}^n \alpha_i y_i x_i$$

$$\frac{\partial L}{\partial b} = 0 \Rightarrow \sum_{i=1}^n \alpha_i y_i = 0$$

Primal and Dual Form of SVM

Primal Problem:

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1, \quad i = 1, \dots, n$$

Dual Problem:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i^T x_j) \\ \text{s.t.} \quad & \alpha_i \geq 0, \quad \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

Key Insight: Only $\alpha_i > 0$ contribute $\Rightarrow$ Support Vectors

Kernel Trick: Motivation

Problem:

- Some datasets are not linearly separable in original space

Idea:

$\phi(x) : x \rightarrow$ higher-dimensional feature space

Goal:

- Make data linearly separable in transformed space

Challenge: Explicit computation of $\phi(x)$ is expensive.

Kernel Trick: Core Idea

Instead of computing $\phi(x)$ explicitly:

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

Key Idea:

- Compute inner product in high-dimensional space implicitly
- Avoid explicit transformation

Result:

- Nonlinear boundary in original space
- Linear separation in feature space

Kernel Trick: Intuition Example

Non-linearly separable data:

- Class +1: points inside circle
- Class -1: points outside circle

Feature mapping:

$$\phi(x_1, x_2) = (x_1^2, x_2^2, x_1 x_2)$$

Idea: Data becomes linearly separable in higher dimension.

Common Kernel Functions

Linear Kernel

$$K(x_i, x_j) = x_i^T x_j$$

Polynomial Kernel

$$K(x_i, x_j) = (x_i^T x_j + c)^d$$

RBF (Gaussian) Kernel

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

Interpretation:

- Linear $\rightarrow$ simple boundary
- Polynomial $\rightarrow$ curved boundary
- RBF $\rightarrow$ highly flexible nonlinear boundary

$$f(x) = \sum_{i \in SV} \alpha_i y_i K(x_i, x) + b$$

Key Points:

- Only support vectors contribute
- Kernel replaces dot product
- Enables nonlinear classification efficiently

Main Idea: Kernel trick enables high-dimensional learning without explicit computation.

SVM Training Procedure (Intuition)

Initialize:

$$w = 0, \quad b = 0$$

For each (x_i, y_i) :

$$y_i(w^T x_i + b)$$

Case 1: Correct classification (≥ 1)

$$w \leftarrow w - \eta(2\lambda w)$$

Case 2: Misclassified (< 1)

$$w \leftarrow w - \eta(2\lambda w - Cy_i x_i)$$

$$b \leftarrow b + \eta y_i$$

SVM Classification Example

Consider two classes:

$$+1 : (3, 3), (4, 3)$$

$$-1 : (1, 1), (2, 1)$$

Assume the separating hyperplane is

$$w = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad b = -4$$

Decision function:

$$f(x) = w^T x + b$$

Class prediction:

$$\hat{y} = \text{sign}(f(x))$$

SVM Classification Example

For point (3, 3):

$$f(3, 3) = 1(3) + 1(3) - 4 = 2$$

$$\hat{y} = +1$$

For point (1, 1):

$$f(1, 1) = 1(1) + 1(1) - 4 = -2$$

$$\hat{y} = -1$$

Since the signs match the true labels, the points are correctly classified.

Distance from Hyperplane

Distance of a point from the hyperplane:

$$d = \frac{|w^T x + b|}{\|w\|}$$

For point (3, 3):

$$d = \frac{|2|}{\sqrt{1^2 + 1^2}} = \frac{2}{\sqrt{2}} = 1.414$$

For point (1, 1):

$$d = \frac{|-2|}{\sqrt{2}} = 1.414$$

Both points are equally distant from the boundary.

Margin Calculation

Suppose the support vectors satisfy

$$w^T x + b = 1$$

and

$$w^T x + b = -1$$

The margin is

$$\text{Margin} = \frac{2}{\|w\|}$$

Since

$$\|w\| = \sqrt{1^2 + 1^2} = \sqrt{2}$$

we obtain

$$\text{Margin} = \frac{2}{\sqrt{2}} = \sqrt{2} = 1.414$$

What Does SVM Optimize?

SVM searches for w and b that maximize the margin.

$$\max \frac{2}{\|w\|}$$

Equivalent optimization:

$$\min \frac{1}{2} \|w\|^2$$

subject to

$$y_i(w^T x_i + b) \geq 1$$

Key Idea:

A larger margin generally leads to better generalization on unseen data.

k-Nearest Neighbors (k-NN)

Definition

- k-Nearest Neighbors (k-NN) is a **supervised learning** algorithm.
- It predicts the class (or value) of a new sample by examining its **k closest training samples**.
- It is a **non-parametric** and **instance-based (lazy)** learning algorithm.

Applications

- Classification
- Regression
- Pattern recognition
- Recommendation systems

How k-NN Works

Algorithm

- 1 Choose the number of neighbors (k).
- 2 Compute the distance between the query point and every training sample.
- 3 Select the k nearest neighbors.
- 4 Predict:
 - **Classification:** Majority voting.
 - **Regression:** Average of neighbors.

Key Idea

“Similar data points tend to have similar labels.”

Distance Measures in k-NN

Euclidean Distance ($p = 2$)

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Manhattan Distance ($p = 1$)

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

Other Distance Metrics

- **Minkowski:**

$$d(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}}$$

- **Cosine Distance:**

$$1 - \frac{x \cdot y}{\|x\| \|y\|}$$

- Suitable for high-dimensional and sparse data (e.g., text).

Important

Distance-based algorithms are sensitive to feature scales. Always normalize or standardize features before applying k-NN.

Choosing the Value of k

Small k (e.g., $k = 1, 3$)

- Sensitive to noise
- Complex decision boundary
- May overfit

Large k (e.g., $k = 15, 21$)

- Less sensitive to noise
- Smooth decision boundary
- May underfit

Practical Tip

Choose an odd value of k (e.g., 3, 5, 7) for binary classification to reduce ties.

Advantages and Limitations of k-NN

Advantages

- Easy to understand and implement
- No explicit training phase
- Naturally handles multi-class problems
- Effective for small datasets

Limitations

- Slow prediction for large datasets
- Sensitive to irrelevant features
- Requires feature scaling
- Performance depends on the choice of k

Training and Prediction in k-NN

Training Phase

- Store all training samples.
- No model is learned.

Prediction Phase

- Compute distances to all training samples.
- Find the nearest neighbors.
- Predict using majority voting (classification) or averaging (regression).

Why is k-NN called a Lazy Learner?

It postpones learning until a prediction is requested.

Effect of Different Values of k

k	Effect
1	Very flexible, sensitive to noise
3–7	Good balance between bias and variance
Large	Stable predictions but may miss local patterns

Rule of Thumb

Small $k \rightarrow$ Overfitting Large $k \rightarrow$ Underfitting

Example of k-Nearest Neighbors (k-NN)

Training Data

Fruit	Weight (g)	Sweetness	Class
A	150	8	Apple
B	170	7	Apple
C	140	9	Apple
D	300	3	Orange
E	320	2	Orange
F	280	4	Orange

A new fruit has

$$X = (180, 6)$$

Goal: Predict whether the new fruit is an **Apple** or an **Orange** using k-NN.

Example: Classification using $k = 3$

Step 1: Compute Euclidean Distance

$$d(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$

Fruit	Distance	Class
A	30.07	Apple
B	10.05	Apple
C	40.11	Apple
D	120.04	Orange
E	140.06	Orange
F	100.02	Orange

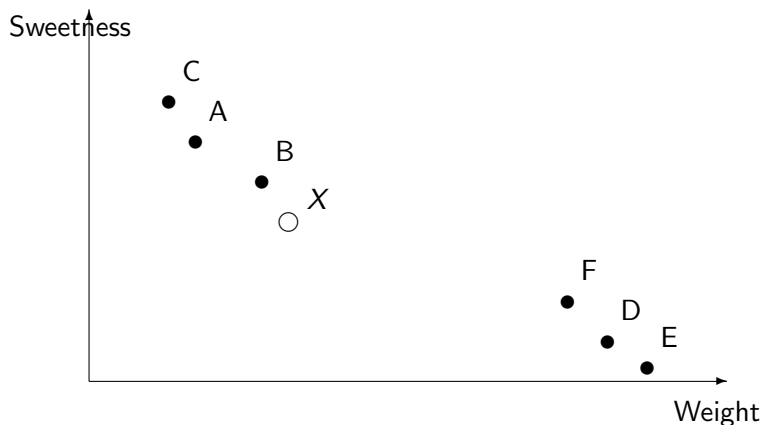
Step 2: Select the 3 nearest neighbors

B, A, C

All three neighbors belong to the **Apple** class.

Prediction = Apple

Visual Illustration



Observation

- The query point X is much closer to Apple samples.
- Therefore, the majority vote predicts: **Apple**

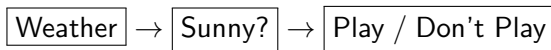
- A **supervised machine learning algorithm** used for both **classification** and **regression**.
- Learns a set of **if–then decision rules** by recursively splitting the dataset into smaller and more homogeneous subsets.
- Represents the decision process as a **tree structure**, making the model easy to understand and interpret.
- At each node, the algorithm selects the feature that best separates the data using criteria such as **Information Gain**, **Gini Index**, or **Variance Reduction**.
- The final prediction is obtained by following a path from the root node to a leaf node.

Structure of a Decision Tree

Components of a Decision Tree

- **Root Node:** Represents the entire dataset and performs the first split.
- **Internal Node:** Represents a test on a feature (e.g., Age > 30?).
- **Branch:** Represents the outcome of a test.
- **Leaf Node:** Represents the final predicted class or regression value.

Example



Entropy measures the amount of **uncertainty** or **impurity** in a dataset.

$$Entropy(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

where

- p_i = probability of class i
- c = total number of classes

Interpretation

- Entropy = 0 : All samples belong to one class (pure node).
- Higher entropy : Classes are mixed (impure node).
- Maximum entropy occurs when all classes are equally likely.

Information Gain

Information Gain measures how much a feature **reduces entropy** after splitting the dataset.

$$IG(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

where

- S = original dataset
- A = selected feature
- S_v = subset after splitting on feature value v

Decision Rule

- Compute Information Gain for every feature.
- Choose the feature with the **highest Information Gain**.
- Repeat the process recursively until a stopping criterion is met.

Decision Tree Construction Algorithm

Steps

- 1 Start with the entire training dataset at the root node.
- 2 Compute the impurity (Entropy or Gini Index).
- 3 Calculate the Information Gain for each feature.
- 4 Select the feature with the highest Information Gain.
- 5 Split the dataset into child nodes.
- 6 Repeat the process recursively for each child node.
- 7 Stop when all samples belong to one class or another stopping criterion is satisfied.

Example: Constructing a Decision Tree

Consider the following training dataset.

Sample	Weather	Play
1	Sunny	No
2	Sunny	No
3	Sunny	Yes
4	Rainy	Yes
5	Rainy	Yes
6	Rainy	No
7	Overcast	Yes
8	Overcast	Yes

Goal: Select the best feature to become the **root node** using Information Gain.

Step 1: Compute Entropy of the Dataset

There are

5 Yes, 3 No

Therefore,

$$p(\text{Yes}) = \frac{5}{8}, \quad p(\text{No}) = \frac{3}{8}$$

The entropy of the dataset is

$$\text{Entropy}(S) = -\frac{5}{8} \log_2 \frac{5}{8} - \frac{3}{8} \log_2 \frac{3}{8}$$

$$\text{Entropy}(S) = -(0.625)(-0.678) - (0.375)(-1.415)$$

$$\boxed{\text{Entropy}(S) = 0.954}$$

Step 2: Compute Entropy After Splitting on Weather

Weather	Yes	No	Entropy
Sunny	1	2	0.918
Rainy	2	1	0.918
Overcast	2	0	0.000

Weighted entropy after splitting:

$$\begin{aligned} Entropy_{\text{split}} &= \frac{3}{8}(0.918) + \frac{3}{8}(0.918) + \frac{2}{8}(0) \\ &= 0.344 + 0.344 + 0 \\ &= \boxed{0.688} \end{aligned}$$

Step 3: Compute Information Gain

Information Gain is

$$IG(S, \text{Weather}) = Entropy(S) - Entropy_{\text{split}}$$

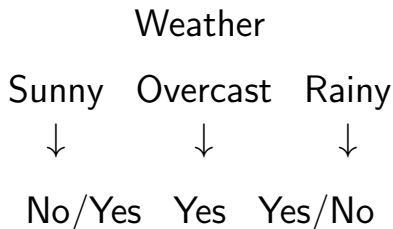
Substituting the values,

$$IG = 0.954 - 0.688$$

$$IG(S, \text{Weather}) = 0.266$$

Since **Weather** has the highest Information Gain, it is selected as the **root node** of the decision tree.

Decision Tree Construction



Interpretation

- Overcast is a pure node ($Entropy = 0$), so it becomes a leaf node labelled **Play = Yes**.
- Sunny and Rainy are still impure.
- These nodes are recursively split using another feature (e.g., Humidity or Wind).

Example: Constructing a Decision Tree

Construct a decision tree using the following training dataset.

ID	Weather	Temperature	Wind	Play
1	Sunny	Hot	Weak	No
2	Sunny	Hot	Strong	No
3	Sunny	Mild	Weak	Yes
4	Sunny	Cool	Weak	Yes
5	Overcast	Hot	Weak	Yes
6	Overcast	Cool	Strong	Yes
7	Rainy	Mild	Weak	Yes
8	Rainy	Cool	Weak	Yes
9	Rainy	Cool	Strong	No
10	Rainy	Mild	Strong	No

Goal: Build a Decision Tree using the Information Gain criterion.

Step 1: Compute the Root Node

There are

6 Yes, 4 No

Therefore,

$$p(\text{Yes}) = \frac{6}{10}, \quad p(\text{No}) = \frac{4}{10}$$

The entropy of the complete dataset is

$$\begin{aligned} \text{Entropy}(S) &= -\frac{6}{10} \log_2 \frac{6}{10} - \frac{4}{10} \log_2 \frac{4}{10} \\ &= 0.971 \end{aligned}$$

$\text{Entropy}(S) = 0.971$

Next, compute the Information Gain for each feature.

Step 2: Split on Weather

Weather	Yes	No	Entropy
Sunny	2	2	1.000
Overcast	2	0	0.000
Rainy	2	2	1.000

Weighted entropy

$$\begin{aligned} Entropy_{split} &= \frac{4}{10}(1) + \frac{2}{10}(0) + \frac{4}{10}(1) \\ &= 0.8 \end{aligned}$$

Information Gain

$$IG(Weather) = 0.971 - 0.8 = 0.171$$

Assume Weather has the highest Information Gain.

Root Node = Weather

Step 3: Split the Sunny Node

Sunny subset

Temperature	Wind	Play
Hot	Weak	No
Hot	Strong	No
Mild	Weak	Yes
Cool	Weak	Yes

Notice that

- Temperature = Hot $\rightarrow$ No
- Temperature = Mild $\rightarrow$ Yes
- Temperature = Cool $\rightarrow$ Yes

Temperature perfectly separates the data.

Sunny $\rightarrow$ Temperature

Step 4: Split the Rainy Node

Rainy subset

Temperature	Wind	Play
Mild	Weak	Yes
Cool	Weak	Yes
Cool	Strong	No
Mild	Strong	No

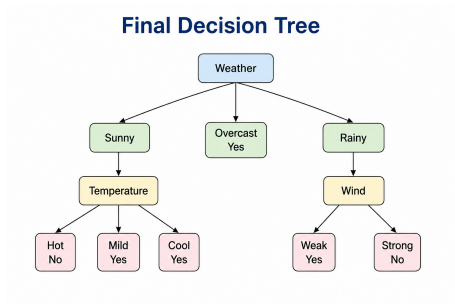
Using Wind,

Wind	Yes	No
Weak	2	0
Strong	0	2

Wind produces pure child nodes.

Rainy $\rightarrow$ Wind

Final Decision Tree



Decision Rules

- If Weather = Overcast $\rightarrow$ Play = Yes
- If Weather = Sunny and Temperature = Hot $\rightarrow$ No
- If Weather = Sunny and Temperature = Mild/Cool $\rightarrow$ Yes
- If Weather = Rainy and Wind = Weak $\rightarrow$ Yes
- If Weather = Rainy and Wind = Strong $\rightarrow$ No

Advantages and Limitations of Decision Trees

Advantages

- Easy to understand and interpret.
- Requires little data preprocessing.
- Handles both numerical and categorical data.
- Captures non-linear relationships.
- Feature importance can be identified.

Limitations

- Prone to overfitting.
- Sensitive to small changes in training data.
- Can become deep and complex.
- May be biased toward features with many categories.
- Lower accuracy than ensemble methods.

What is Naïve Bayes?

Naïve Bayes is a supervised machine learning algorithm based on **Bayes' Theorem**. It predicts the class of a new sample by calculating the probability that the sample belongs to each class.

Key Characteristics

- Probabilistic classifier that predicts using probabilities.
- Uses Bayes' theorem to estimate posterior probabilities.
- Assumes all features are **conditionally independent** given the class label (the "naïve" assumption).
- Very fast to train and predict, making it suitable for large and high-dimensional datasets.
- Particularly effective for text mining and document classification.

Bayes' Theorem

Bayes' theorem updates our belief about an event after observing new evidence.

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

where

- $P(A|B)$: **Posterior** – Probability of event A after observing B .
- $P(B|A)$: **Likelihood** – Probability of observing B if A is true.
- $P(A)$: **Prior** – Initial probability of event A .
- $P(B)$: **Evidence** – Overall probability of observing B .

Goal: Update prior knowledge using observed data.

Naïve Bayes Classification Rule

Suppose a data sample has features

$$X = (x_1, x_2, \dots, x_n)$$

Using Bayes' theorem,

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}$$

where

- $P(C)$: Prior probability of class C
- $P(X|C)$: Likelihood of observing features X
- $P(C|X)$: Posterior probability
- $P(X)$: Evidence (same for all classes)

The predicted class is

$$\hat{C} = \arg \max_C P(C|X)$$

Decision Rule: Choose the class with the highest posterior probability.

Conditional Independence Assumption

The word "**Naïve**" comes from assuming that all features are independent given the class.

Instead of computing

$$P(x_1, x_2, \dots, x_n | C),$$

Naïve Bayes assumes

$$P(X|C) = \prod_{i=1}^n P(x_i|C).$$

Advantages of this assumption

- Simplifies probability computation.
- Reduces computational complexity.
- Requires fewer training samples.

Note: Although the independence assumption is often unrealistic, Naïve Bayes performs surprisingly well in many real-world problems.

Naïve Bayes Example

Problem: Predict whether a person will **Buy a Computer** based on Age.

Age	Buy Computer
Young	Yes
Young	Yes
Young	No
Middle	Yes

Age	Buy Computer
Middle	Yes
Senior	No
Senior	No
Senior	Yes

Suppose a new customer has:

Age = Young

We calculate:

$$\begin{aligned}P(\text{Yes}) &= \frac{5}{8}, & P(\text{No}) &= \frac{3}{8} \\P(\text{Young}|\text{Yes}) &= \frac{2}{5}, & P(\text{Young}|\text{No}) &= \frac{1}{3} \\P(\text{Yes}|\text{Young}) &\propto P(\text{Yes})P(\text{Young}|\text{Yes}) = \frac{5}{8} \times \frac{2}{5} = 0.25 \\P(\text{No}|\text{Young}) &\propto P(\text{No})P(\text{Young}|\text{No}) = \frac{3}{8} \times \frac{1}{3} = 0.125\end{aligned}$$

$\therefore$ Prediction = Yes

Types of Naïve Bayes

Type	Suitable Data
Gaussian Naïve Bayes	Continuous numerical features following a Gaussian distribution.
Multinomial Naïve Bayes	Count data such as word frequencies in documents.
Bernoulli Naïve Bayes	Binary features (0/1 or Yes/No).

Common Applications

- Spam email detection
- Sentiment analysis
- News and document classification
- Medical diagnosis

Advantages of Naïve Bayes

- Simple to understand and implement.
- Very fast training and prediction.
- Works well with high-dimensional datasets.
- Performs well even with relatively small training data.
- Naturally supports multi-class classification.
- Widely used for text classification and spam filtering.

Limitations of Naïve Bayes

- Strong independence assumption may not hold in practice.
- Performance decreases when features are highly correlated.
- Zero-frequency problem: If a feature never appears in the training data for a class, the probability becomes zero.
- Usually handled using **Laplace (Add-One) Smoothing**.
- Probability estimates may not always be well calibrated, although classification accuracy is often good.

Summary

Property	k-NN	Decision Tree	Naïve Bayes
Type	Instance-based	Tree-based	Probabilistic
Training Speed	Fast	Moderate	Very Fast
Interpretability	Low	High	Moderate
Handles Nonlinearity	Yes	Yes	Limited
Suitable for Text Data	No	Moderate	Excellent